



UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master's Degree in Computer Science

AMM Scheduler:

Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

Prof. Marco Danelutto

Candidate:

Simone Stanganini

ACADEMIC YEAR 2024/2025

Contents

1	Introduction	5
1.1	Contest	5
1.2	Objectives	6
1.3	Thesis Structure	6
2	Tools and Technologies	7
2.1	Experimental Platform	7
2.1.1	The SoC: Intel Core Ultra 9 185H	7
2.1.2	NVIDIA GeForce RTX 4060	8
A	Questa è un'appendice	9

Chapter 1

Introduction

1.1 Contest

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds. Today, the approach has changed completely, shifting towards hardware specialization.

Modern computers, from servers to embedded SoCs, don't rely on a single processor anymore. Instead, they integrate different specific accelerators:

- **CPU**: Optimized for sequential tasks and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI.

This variety of hardware has created a gap with the software. While hardware offers different resources for different workloads, traditional software tends to be static. It usually assigns tasks to a single device (often just the CPU or GPU) and ignores the other available resources.

The result is an inefficient use of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed, so the current challenge is *orchestration*: we need a system that can dynamically map each task to the right accelerator, using the entire heterogeneous topology of the system.

1.2 Objectives

The main goal of this thesis was to design and implement a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase. I focused on integrating and studying different compute backends, such as CUDA, SYCL, OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler. The core of the project focused on the asymmetric orchestration logic, designed to decide in real-time which resource to use for each specific workload.

Finally, the system was validated with experimental results on a complex hybrid architecture. This setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of identical matrices, streams of heterogeneous matrices, and splitting larger tasks across multiple accelerators.

1.3 Thesis Structure

The work is organized into the following chapters:

- **Chapter 2:** Introduces the hardware technologies (CPU, GPU, NPU) and the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS).
- **Chapter 3:** Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4:** Details the code implementation. It covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting.
- **Chapter 5:** Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6:** Draws conclusions on the work done and suggests possible future developments.

Chapter 2

Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate the system. I chose to work on a hybrid and highly heterogeneous architecture to test the scheduler in a real-world scenario. In this scenario, devices with different performance characteristics and programming models must work together.

2.1 Experimental Platform

All development and benchmarking activities were carried out on a mobile workstation (Lenovo Yoga Pro 9). This platform represents a modern high-performance "Edge" computing node well, it integrates all the types of accelerators studied into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU. Having these components available at the same time allowed me to emulate, within a single node, the typical orchestration issues of heterogeneous distributed systems.

2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:

- **Host CPU:** This cpu is made of 16 hybrid cores (6 Performance-cores for raw power, 8 Efficient-cores, and 2 Low-Power-cores for background efficiency). In this project, this unit acts as the Host. It runs the scheduler code, manages task dispatching, and coordinates data flows to the accelerators.

- **NPU** - Intel AI Boost: The Ultra 9 includes a native Neural Processing Unit. This is a low-power accelerator designed specifically for neural network inference. In my framework, the NPU is used via the OpenVINO backend to run AI workloads without loading the GPU or the main CPU.
- **iGPU** - Intel Arc Graphics: : Integrated into the SoC, this graphics unit (based on Xe-LPG architecture) offers parallel computing capabilities with direct access to system memory. Even though it is less powerful than a discrete solution, it is a key target for testing the SYCL backend in a unified memory scenario.

2.1.2 NVIDIA GeForce RTX 4060

To complement the Intel SoC, the system is equipped with a discrete NVIDIA GeForce RTX 4060 GPU Laptop Edition, based on the Ada Lovelace architecture, this card acts as the high-throughput accelerator of the system.

Unlike the iGPU, it has dedicated video memory (GDDR6 VRAM) and specific cores (CUDA Cores). Within the scheduler, this device is managed via the CUDA backend. It is typically chosen for processing massive data streams or large matrices, where the execution speed makes up for the cost of data transfer over the PCIe bus.

Questa è una citazione [1].

Appendix A

Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Bibliography

- [1] Nome Cognome. Il gabbiano e altri incredibili racconti impegnati ma non troppo. *Il mio giornale*, 1:69, 04 2022.